Computers, Environment and Urban Systems 81 (2020) 101473

Contents lists available at ScienceDirect

Computers, Environment and Urban Systems

journal homepage: www.elsevier.com/locate/ceus

GSAM: A deep neural network model for extracting computational representations of Chinese addresses fused with geospatial feature

Liuchang Xu[a], Zhenhong Du[a,b,*], Ruichen Mao[a], Feng Zhang[a,b], Renyi Liu[a,b]

[a] School of Earth Sciences, Zhejiang University, 38 Zheda Road, Hangzhou 310027, China
[b] Zhejiang Provincial Key Laboratory of Geographic Information Science, 148 Tianmushan Road, Hangzhou 310028, China

ARTICLE INFO

Keywords:
Chinese addresses
Computational representations
Semantic/geospatial fusion
Bidirectional encoder representations from transformer
Feature fusion clustering
Address location prediction task

ABSTRACT

Addresses are one of the most important geographical reference systems in natural languages. In China, due to the relatively backward address planning, there are a large number of non-standard addresses. This kind of unstructured text makes the management and application of Chinese addresses much more difficult. However, by extracting the computational representations of addresses, it can be structured and its related applications can be extended more conveniently. Therefore, this paper utilizes a deep neural language model from natural language processing (NLP) to automatically extract computational representations through an unsupervised address language model (ALM), which is trained in an unsupervised way and is suitable for a large-scale address corpus. We propose a solution to fuse addresses and geospatial features and construct a geospatial-semantic address model (GSAM) that supports a variety of downstream tasks. Our proposed GSAM constructing process consists of three phases. First, we build an ALM using bidirectional encoder representations from Transformers (BERT) to learn the addresses' semantic representations. Then, the fusion clustering results of the semantic and geospatial information are obtained by a high-dimensional clustering algorithm. Finally, we construct the GSAM based on the fused clustering results using novel fine-tuning techniques. Furthermore, we apply the extracted computational representation from GSAM to the address location prediction task. The experimental results indicate that the target task accuracy of the ALM is 90.79%, and the result of semantic geospatial fusion clustering strongly correlates with fine-grained urban neighbourhood area division. The GSAM can accurately identify clustering labels and the values of evaluation metrics are all above 0.96. We also demonstrate that our model outperforms purely ALM-based and word2vec-based models by address location prediction task.

1. Introduction

Addresses are in a natural language and are frequently the results of subjective interpretation and creation guided by people living in these places (Conedera, Vassere, Neff, Meurer, & Krebs, 2007). Unlike common natural languages, addresses also contain geospatial attributes, and most studies involving addresses typically correlate geospatial data (Goldberg, Wilson, & Knoblock, 2007). Therefore, it is a significant challenge for human beings to capture the cognitive concept of addresses and establish a connection between addresses and geospatial attributes (e.g., coordinates or geometric relations) that can be understood by computers.

In China, due to the rapid development of cities, address planning and management are very chaotic. This disorganization also leads to relevant government departments collecting many non-standard addresses due to the habits of different address collectors and their familiarity with the place of interest. In view of the above problems, there have been various studies on Chinese address resolution and standardization (Guo, Zhu, Guo, Zhang, & Su, 2009; Zhu, Zhao, Liu, & Zhao, 2018). In contrast to previous studies, we use language modelling to pre-train the address corpus from the perspective of semantic understanding. Through unsupervised training, it can automatically extract semantic computational representations.

On the other hand, in previous studies, there exists the problem that addresses and geospatial locations cannot be deeply fused. Relevant studies include address similarity matching, geographic information retrieval, address location prediction, entity fuzzy matching, geocoding error detection, etc.(Cayo & Talbot, 2003; Goldberg et al., 2007; Jones & Purves, 2009; Ju et al., 2016; Kim, Vasardani, & Winter, 2017; Matci & Avdan, 2018; Tian et al., 2016), they are still tweaking on unstructured textual data, which are correspondingly lacking in generality. Deep learning is a novel branch of machine learning that has

* Corresponding author at: School of Earth Sciences, Zhejiang University, Xixi Campus, Hangzhou, Zhejiang, China.
E-mail addresses: xlczju@zju.edu.cn (L. Xu), duzhenhong@zju.edu.cn (Z. Du).

https://doi.org/10.1016/j.compenvurbsys.2020.101473
Received 3 July 2019; Received in revised form 27 January 2020; Accepted 3 February 2020

been broadly applied in geospatial research and significant progress has been made in language model, making it possible to apply NLP's methods to addresses computational representations extraction. However, the current universal language model does not contain the geospatial characteristics of addresses. Therefore, we integrate the semantic and geospatial representations of addresses to build a GSAM and use transfer learning in deep learning for reference to make the GSAM more widely applicable.

Considering these difficulties, this paper designs a semantic and geospatial feature fusion architecture for addresses based on deep neural network language modelling. Our main contributions are as follows:

1. Using a neural network based on Transformer encoders with multi-head self-attention and a target task such as a cloze task, the address language model (ALM) is trained in an unsupervised way and is suitable for a large-scale address corpus. (Section 3.2)
2. The fusion method of address computational representations and geographical coordinates is designed and clustering results that are strongly correlated with fine-grained urban neighbourhood area division are obtained. (Sections 3.3.1–3.3.3)
3. Referring to the fine-tuning theory, pre-trained ALM is fine-tuned based on the fused clustering results to build a GSAM so that the model has both semantic and geospatial features. (Section 3.3.4)
4. Taking the GSAM as the upstream model, the downstream verification task of address coordinate prediction is designed. The target task downstream structure was constructed as three fully connected neural networks. We verify that the GSAM-based model has the highest accuracy. (Section 3.4)

2. Related work

Language modelling is gaining momentum in research including text mining and location prediction. This paper is inspired by 2 lines of research: extracting the semantic representations of addresses through a neural language model and integrating text information and geographic locations using a language model. In this section, we compare studies that have used different language models based on deep neural network, and then discuss the research gap in fusing the semantic and geospatial feature.

2.1. Language model based on a deep neural network

Language modelling refers to modelling by learning the joint probability of word sequences in a language. The neural language model was first proposed by Bengio, Ducharme, Vincent, and Jauvin (2003), who proposed a method to represent each word as a dense vector and constructed a feedforward neural network structure to learn the distributed representation of each word. Compared with the traditional statistical language model, distributed representation learning provides a data-driven way to calculate semantic similarities and is more generalizable. With the rise of deep learning methods, Mikolov, Sutskever, Chen, Corrado, and Dean (2013) applied convolutional neural networks to language models. However, these models, which aim to generate static word representations, cannot handle polysemy. For this reason, the concept of a pre-trained language model was proposed (Dai & Le, 2015). Peters et al. (2018) have proved that these models perform well on various tasks. They used the language model as the feature of the target model and built a new deep contextualized word representation. They adopted a two-layer bidirectional LSTM (i.e., Long short-term memory) network structure and set the target task to predict words based on the context. Later, Radford, Narasimhan, Salimans, and Sutskever (2018) proposed the GPT (i.e., Generative Pre-Training) language model, which used Transformer-based networks to extract semantic features with good parallelism and long-distance capture effects. In addition, a fine-tuning mode was also proposed to help complete the downstream tasks. Since the GPT model is unidirectional, Devlin, Chang, Lee, and Toutanova (2018) proposed a bidirectional language model based on Transformer networks that integrates the advantages of the above two language models. Therefore, we mainly refer to the network architecture of Devlin's model when building the ALM.

2.2. Using language modelling to fuse text and location information

The general idea of injecting geographic information into language model has been partially explored. Some common studies have focused on location prediction based on social media text data that use language modelling methods. Wing and Baldridge (2011) were the first to use statistical language modelling to automatically localize the geotagged text data from Wikipedia and Twitter. They used n-gram statistical language model and a discrete, regular grid division of the Earth's surface to predict the grid belonging to the document to achieve the predicted location origin of the text. However, the region of their study covered the whole United States, which led to an uneven data distribution due to the uniform grid divisions. In a follow-up study, they used a k-d-tree to build an adaptive grid to improve the prediction results (Roller, Speriosu, Rallapalli, Wing, & Baldridge, 2012). To improve the accuracy of the position predictions, Laere, Schockaert, Tanasescu, Dhoedt, and Jones (2014) integrated multi-source language models and used k-medoid clustering to extract features from text. In addition, DeLozier, Baldridge, and London (2015) calculated the geographical distribution of each word based on the local spatial statistics of a set of geo-referenced language models and obtained better results without a gazetteer. In contrast to the above research, the regional granularity of our research is at the city level, which makes the text more densely geographically distributed and has more regions that conform to human experience. Accordingly, we abandon the method of assigning geospatial tags based on meshing or spatial clustering and instead use a method of fusing address semantic and geospatial features.

On the other hand, an increasing number of researchers combine textual representations and geographic information, such as for urban functional region extraction (Zhai et al., 2019), measuring POIs' spatial context patterns (Liu, Andris, & Rahimi, 2019), predictions of future POI visits (Feng, Cong, An, & Chee, 2017), recommended POIs (Hao, Cheang, & Chiang, 2019)and other urban space studies. Cocos and Callison-Burch (2017) used geospatial information from OpenStreetMap and Google Places data as context to train geospatial word embedding, indicating that the geospatial context does encode information about semantic relatedness. Yan, Janowicz, Mai, and Gao (2017) used a distance-binned, information-theoretic approach to augment the spatial context and further integrated semantic and spatial information. However, the above studies changed the semantic features by processing the spatial information and then formed the expression method for semantic features. In our work, semantic features and geospatial coordinates are directly fused and are expressed by newly generated fusion features.

3. Methodology

3.1. Overall architecture

In this study, we develop a three-stage computational framework. Fig. 1 displays the overall architecture of this framework.

1. Use a Chinese address corpus to pre-train the address language model (ALM) based on bidirectional encoder representations from Transformers (BERT).
2. Coordinates are weighted to obtain the distribution of the semantic/geospatial fusion, and the improved k-means algorithm is used to cluster the fusion features. Fine-tune the ALM to construct the

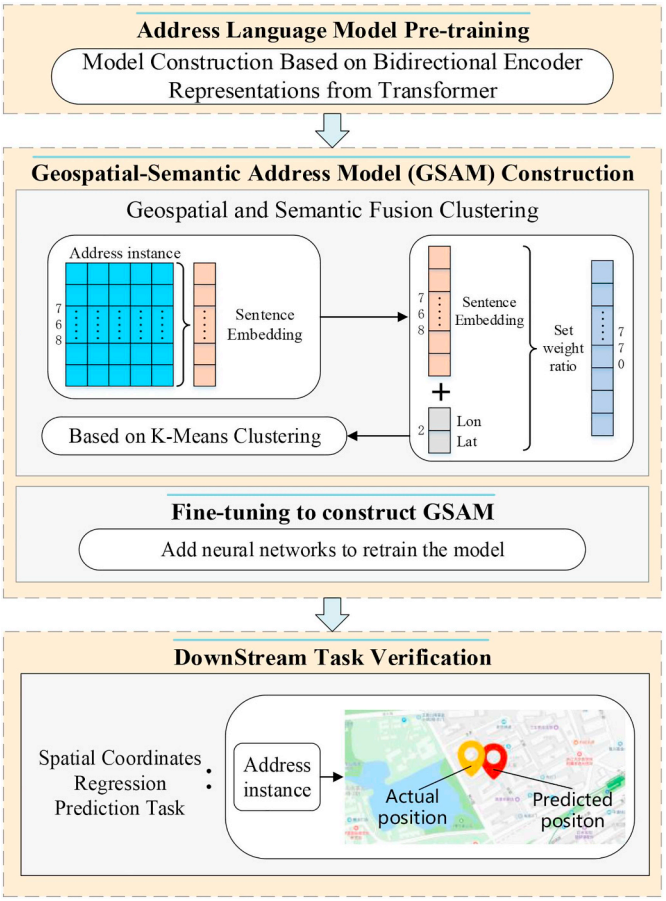

Fig. 1. Overall architecture of the proposed study framework.

geospatial semantic address model (GSAM) based on the clustering results.

3. Downstream task of address coordinate prediction is designed to verify the validity of the GSAM.

3.2. Address language model pre-training based on BERT

This section uses an address corpus to pre-train the ALM. The ALM's neural network architecture is shown in Fig. 2.

First, we tokenize the raw addresses. Tokenization in our research is the process of tokenizing or splitting an address into a list of tokens. When handling word-level tokenization of non-logo syllabary languages, such as English, it generally has three main steps: 1. Text normalization: convert all whitespace characters to spaces and lowercase the input and strip out accent markers. 2. Punctuation splitting: split all punctuation characters on both sides (i.e., add whitespace around all punctuation characters). 3. WordPiece tokenization: apply whitespace tokenization to the output of the above procedure and apply WordPiece (i.e., an unsupervised multi-lingual text tokenizer) tokenization to each token separately. However, Chinese address does not contain whitespace and its dictionary is composed of different single characters, so we add whitespace around every character as character-level tokenization at the input stage. The purpose of the token makes basic units (i.e., Chinese characters and punctuation marks) clearly segregated, which is a key step in pre-processing for our model input.

Moreover, the embedded positions are added to the initial input to record the location information. Therefore, a given token's input representation is constructed by summing the corresponding token and position embeddings. It's worth mentioning that unlike traditional Transformer, where the position is encoded through trigonometric functions (Vaswani et al., 2017), we directly obtained the position

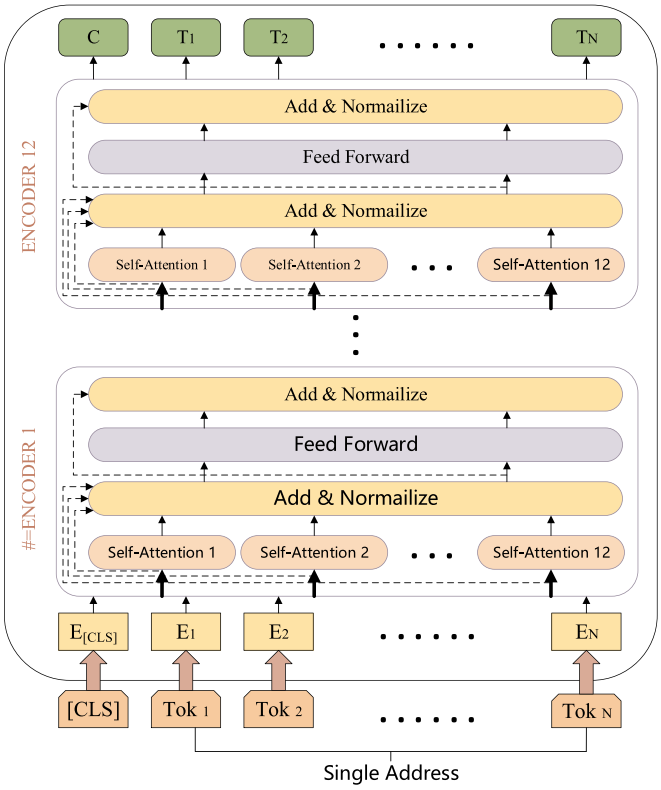

Fig. 2. Address language model neural network architecture.

embeddings through training. As shown in Fig. 2, the ALM adopts a multi-head self-attention mechanism. ALM's model architecture consists of multi-layer Transformer encoders. There are 12 encoders (i.e., Transformer blocks). Each encoder contains 12 self-attention heads. Each self-attention heads are followed by residual connections and layer-normalization. The self-attention mechanism allows each character and all characters in the same address description to calculate their attention to each other regardless of the address description length, which enables each character to retain the global weight information. In Chinese addresses, since the ALM pre-training is based on characters, the semantics of each character in different contexts will be very different. An example of a Chinese address is shown in Fig. 3. The characters highlighted in red are the same but represent a completely different level of the address's hierarchy and meaning. The self-attention mechanism considers the context by allowing the same character to be assigned a higher attention score for the corresponding characters in the corresponding context according to its association with all the other words. Each character in a sentence is represented by three feature vectors: *Query*(Q), *Key*(K), *Value*(V). As can be seen, Q, K and V are shorthand for query, key, and value, respectively. Where K and V are one-to-one correspondence, they are like key-value relations. In the self-attention mechanism, Q, K and V are all input token sequences of encoder (i.e., each Chinese address after tokenization in the paper), and the internal correlation of encoder input token is finally obtained. The essence of the self-attention function can be described as a mapping of a query to a series of key-value pairs. These three feature vectors are calculated by multiplying the word embedding X of the character by the three weight matrices (W^Q, W^K, W^V), which are obtained by neural network training. The formula of attention score is given as follows:

$$Attention(Q,K,V) = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right)V \quad (1)$$

Where d_k is the dimension of the *Key*. The above formula means that, firstly, the dot product of Q and each K is obtained, and a softmax

Fig. 3. Polysemy in Chinese addresses.

layer is used for normalization to get the similarity between *Q* and each *K*. Then we take the weighted sum to get a d_k dimensional vector. $\sqrt{d_k}$ plays a role in regulation, so that the dot product is not too large.

In addition, a multi-head attention mechanism can acquire different subspace semantics for learning more semantic features from the address corpus. It maps the feature vectors that were originally mapped only once and obtains *Q, K,* and *V* in multiple semantic subspaces. Then, an attention value operation was carried out on these vectors, and finally, the results were concatenated.

The ALM uses a pre-training objective task: the "masked language model" (MLM), inspired by cloze tasks. The task masks some characters in the address and then predicts them during pre-training. Unlike left-to-right language model pre-training, the MLM task allows the representation to fuse the left and the right contexts (Devlin et al., 2018). Because Chinese addresses usually have layered, nested relationships, including hierarchical and multi-level granularity relationships, this bidirectional attentional representation pattern is critical. We mask out 15% of the characters in the input with the "[Mask]" symbol, run the entire address sequence through ALM, and the objective is to predict the original vocabulary id of the masked word based only on its context.

3.3. Geospatial-semantic address model (GSAM) construction

In our work, a coordinate is taken as the geospatial feature. If we only use it as a clustering indicator, the result of the division will be contrary to people's understanding of the area of interest. For example, the division of the two residential quarters is shown in the panel on the left in Fig. 4. The distribution of the two residential quarters on the map is semi-surrounded. The panel on the right is the spatial clustering result of two clusters. In order to better display the contrast effect, we have drawn the buffers for the clustering result of points. The comparison shows that the distribution of clustering results does not coincide with the distribution of actual residential quarters and has a great difference, which indicates that different addresses that belong to the same residential quarters will be divided into different categories according to the clustering results. It proves that the pure spatial clustering results are not good. The following subsections list specific solutions to this problem.

3.3.1. Contextual feature representation with fixed length

The total number of features extracted by the ALM is the number of characters contained in one address multiplied by the dimension of the character's vector. Since the length of each address is not fixed, this approach will result in a non-fixed representation length for each address, which is difficult to use as input for subsequent tasks. Thus, pooling is required to obtain a fixed representation of a sentence.

In our work, pooling refers to the reduction of the required fixed dimensions for all contextualized word embeddings in an address. The most popular pooling strategies are average pooling and maximum pooling. We denote the length of the address by *L* and the semantic feature dimension of characters by *S*. The address's semantic representation is stored as *charEmbed*[*L*,*S*]. Average pooling and maximum pooling are expressed as the following formula:

$$\begin{aligned} &avg\_sentEmbed\,[\mathrm{i}] = \mathrm{avg}(charEmbed\,[0\colon \mathrm{L}][\mathrm{i}]) \\ &max\_sentEmbed\,[\mathrm{i}] = \max(charEmbed\,[0\colon \mathrm{L}][\mathrm{i}]) \\ &\text{where:} \mathrm{i} \in [0, \mathrm{S}\text{-}1] \end{aligned} \tag{2}$$

Previous studies showed that average pooling handles all the words in the whole sentence, while max pooling focuses on the keywords and significant representations in the sentence (Er, Zhang, Wang, & Pratama, 2016; Kalchbrenner, Grefenstette, & Blunsom, 2014; Tan, Dos Santos, Xiang, & Zhou, 2016; Tan, Santos, Xiang, & Zhou, 2015). To take advantages of both methods, we take the average and the maximum of the hidden state of the encoding layer on the time axis, add the two, and obtain the fixed length sentence representation that has the same dimension size as the hidden layer.

In addition, we need to consider which encoder's hidden layer output in the ALM is taken as the output. Through relevant research on the BERT language model that has an architecture similar to that of the ALM, it was found that the output of the encoder has the following performance in the task of named entity recognition: concatenate the last four hidden layers > sum the last four hidden layers > others (Devlin et al., 2018). Since concatenation would cause dimensional

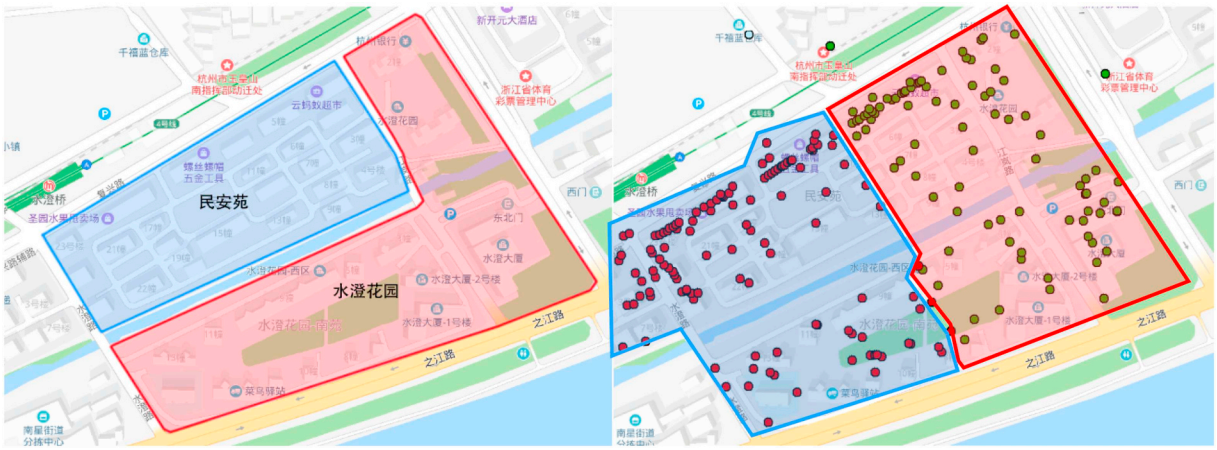

Fig. 4. Comparison of the spatial distribution and the spatial clustering results.

disaster (the hidden layer size is 768 in our work), we sum the last four hidden layers' output.

3.3.2. Spatial features weighted concatenation

Previous studies have shown that feature concatenation is the most popular feature-level fusion methodology (Özay & Vural, 2010; Peters et al., 2018; Zheng, Zhu, Li, Pang, & Wang, 2019). Accordingly, we use feature concatenation to fuse the addresses' semantic and spatial features. The two features need to be dimensioned due to their different magnitudes before concatenated. We propose a dimensionless representation based on range and weight adjustment. Geospatial differences are usually measured by the Euclidean distance, and text similarity can be calculated by the Euclidean distance or the cosine similarity (Wasserman, 2004). For consistency, we use the Euclidean distance as a metric for the semantic and geospatial features and calculate the range of Euclidean distances between them. We denote the number of address semantic features as S and the address corpus as D, and the range between them can be expressed by the following equations:

$$\begin{aligned} sent\_range &= \max\left(\sqrt{\sum_{i=1}^{S}(sentEmbed_{d1\in D}[i] - sentEmbed_{d2\in D}[i])^2}\right) \\ coor\_range &= \max\left(\sqrt{\sum_{i=1}^{2}(coorEmbed_{d1\in D}[i] - coorEmbed_{d2\in D}[i])^2}\right) \end{aligned} \tag{3}$$

where *sent_range* and *coor_range* represent the range of Euclidean distances in the semantic features and geospatial features vector space respectively, *sentEmbed* and *coorEmbed* represent the instance of semantic vector and geospatial vector respectively, and i represents the vector dimension currently calculated.

Then, the approximate ratio between the two range values can be calculated, and we implement the dimensionless operation. The formulas are as follows:

$$\begin{aligned} sentEmbed &= sentEmbed \times \frac{sent\_range}{coor\_range} \times \lambda \\ coorEmbed &= coorEmbed \times (1 - \lambda) \end{aligned} \tag{4}$$

We use λ here to adjust the weight ratio between the two range values.

Finally, we concatenate the semantic and geospatial features into a new fusion representation, which is referred to as *concatEmbed*. The formula is as follows:

$$concatEmbed = \{sentEmbed, coorEmbed\} \tag{5}$$

The above feature fusion method completely retains the semantic and geospatial features of addresses, and the fused feature values only extend to two dimensions on the basis of the semantic representation, which will not cause dimensional disaster in the calculation (Zhou & Bhanu, 2008).

3.3.3. Clustering based on the K-means algorithm

Collections of addresses have a large data volume and a dense spatial distribution. According to the above section, the new fusion representation is high dimensional. Correspondingly, the K-means algorithm has a high time efficiency in practice, which is suitable for big data and high-dimensional clustering applications. Additionally, the number of clusters must be specified in the K-means algorithm, which enables us to adjust the number of clusters to achieve the desired results based on additional reference indicators (e.g., administrative division, population). Accordingly, we choose an improved K-means algorithm for clustering.

The K-means algorithm divides a group of samples into disjoint clusters. Each cluster is described by the mean of the samples in the cluster, which minimizes the sum of squares for all the data in the cluster (MacQueen, 1967). Since the fused representations are dense vectors, we use Elkan's algorithm to improve the efficiency (Elkan, 2003). This algorithm uses the triangle inequality to accelerate K-means, which can reduce the number of unnecessary distance calculations. We also use a variant of K-means clustering, mini-batch K-means clustering, to improve the clustering speed (Sculley, 2010). Each time it updates the cluster centre, it first selects random samples in each training iteration and then updates the cluster centre in the subsets. In addition, for the purpose of making the clustering results as close to the global optimal solution as possible, we use K-means++ for initialization (Arthur & Vassilvitskii, 2007). Its core idea is to make the distance between clustering centres as large as possible when initializing the clustering centres. Finally, we run the algorithm 10 times on random initialization centre points during the experiment, and the clustering result output with the smallest squared error is taken to avoid the dependence of K-means++ on the initial centre point.

3.3.4. Fine-tuning to construct the GSAM

In contrast to the fine-tuning method from previous research, the source of the target data and the source data in our study are the same, that is to say, the source domain and the target domain are consistent; only the annotation information of the target data is different. Based on clustering results obtained in the previous section, a clustering result label is assigned to each address. Then, we construct neural networks for predicting the clustering of addresses. In fine-tuning, we define an encoder-decoder model. The ALM is packaged into a large encoder with an additional decoder structure for fine-tuning.

The neural network of the GSAM is shown in Fig. 5. The network in the decoder is usually based on the specific task. In the classification task, previous studies have shown that the decoder can achieve a good performance as a single linear layer (Devlin et al., 2018). In the ALM pre-training phase, a specific identifier *[CLS]*, which is the representation for the classification task, is added before each address to indicate participation in the training phase. The *[CLS]* takes the output of the last hidden layer from the ALM as the input of the decoder, and we denote it by $C \in \mathbb{R}^H$, where H is the dimension size. When building a decoder, we develop a fully connected layer with tanh activation function, denoted by $U \in \mathbb{R}^{H\times H}$. After the nonlinear transformation of

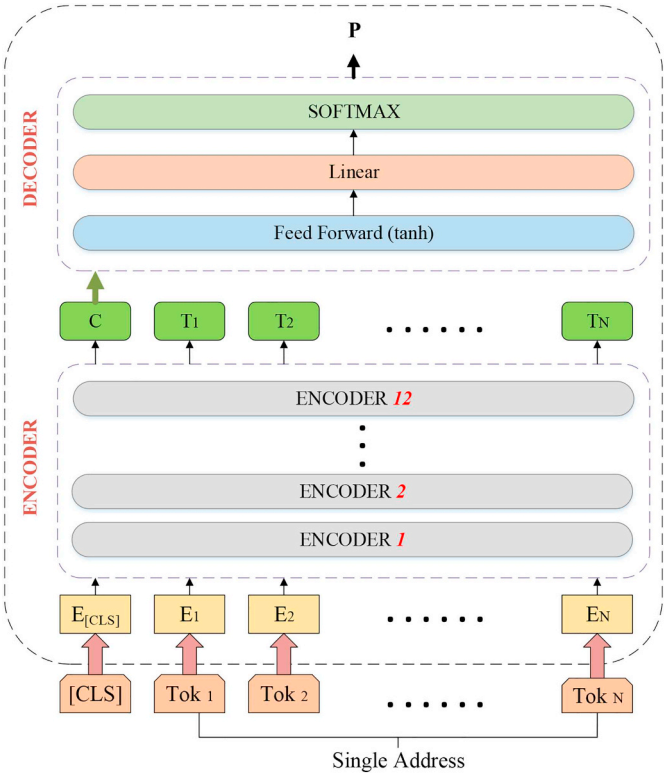

Fig. 5. GSAM network architecture.

the input semantic features in the fully connected layer, the semantic features are transformed into the probability distribution features of classification problems. Batch normalization and dropout are used in the decoder. Then, we add a linear layer with K neurons, where K is the number of classification labels, denoted by $W \in \mathbb{R}^{K \times H}$. Finally, we add a softmax layer to output the probability, denoted by $P \in \mathbb{R}^{K}$.The decoder's loss function is log(P), which makes the value of the correct tag as large as possible.

During fine-tuning, the encoder and the decoder, that is, all the parameters of the neural network, participate in the training, which allows us to take advantage of the information captured by the pre-training model as well as to make changes to the pre-training models to implement specific tasks.

3.4. Address location prediction task methodology

We use the address coordinate prediction task as an example to show the workflow of the standard GSAM's downstream task to guide the application of more downstream tasks. Strictly speaking, the task is still a transfer learning model based on the fine-tuning method, so the design part of this section focuses on the target task module. The target task module should follow three principles: 1. The output is a measure of the geospatial-semantic fusion effect. 2. The neural network should be simple. 3. All neural networks except the target task module are frozen (a detailed analysis in Section 4).

The neural network architecture for spatial coordinate prediction was proposed by Liu and Inkpen (2015), which consists of three fully connected neural layers. Therefore, we develop three fully connected layers as the hidden layers of the target task module and add a linear layer with 2 neurons to the regression task, i.e., the coordinates (latitude, longitude). It is worth mentioning that we project the longitude and latitude of the WGS84 coordinate system of the original data into the CGCS2000 coordinate system, and the projection result is in metres. The loss value can directly display the distance error between the real value and the predicted value. Unlike fine-tuning, the GSAM is completely frozen (i.e., the parameters of the model are fixed and cannot be changed during training) during training, and only the task module is trained. Howard and Ruder (2018a) showed that if all layers of a model are optimally adjusted, the original model will be at risk of catastrophic forgetting. The experimental comparison in Section 4.5 also proves that when the GSAM is not frozen, the predicted results are completely unsatisfactory. On the other hand, Howard and Ruder (2018b) showed that when the dimension size of the model is higher than itself, high-dimensional mapping of the feature information can be achieved, which enables the feature to be expressed in a more comprehensive form. Therefore, we compare the influence of the hidden layer size on the results. We can determine the range of the hidden layer sizes that can help guide the subsequent tasks. For comparison, we also apply a task module to the extracted semantic features based on word2vec.

All fully connected layers in the task use ReLU as the activation function, and each layer applies dropout. The objective function is the Euclidean distance between the predicted and real coordinates. However, the value of the output after the projection is relatively large (millions), e.g., (3,360,322.4412, 1,095,263.519). We subtracted all the converted output values from the corresponding reference values, and make sure that all the values after subtraction are still positive. Let $R = \{(x_r, y_r | r = 1, 2, 3, \ldots, N)\}$ be a set of values of the actual projection points, where N is the total amount of data. We set the reference value as a constant, denoted by X_c, Y_c, and $X_c < = x_r$, $Y_c < = y_r$. Let $P = \{(x_p, y_p | p = 1, 2, 3, \ldots, N)\}$ be a set of predicted values. The definition of the objective function is as follows:

$$L = \sqrt{(x_p - x_r - X_c)^2 + (y_p - y_r - Y_c)^2} \quad (6)$$

4. Experiments and discussion

4.1. Dataset

The raw data are manually collected from government agencies, which all refer to addresses in Shangcheng District, Hangzhou, Zhejiang, China. Shangcheng District covers an area of 26.06 km^2 and has jurisdiction over 6 subdistricts and a total of 54 communities.

The address dataset has multiple attributes, including POIs, road intersections, place names abbreviations, layer upon layer of nested address descriptions, missing hierarchical addresses, and standard addresses. In total, there were 1,984,658 addresses in the raw dataset, which included fields for the address text, the longitude, the latitude, and the community name. The test data are pre-processed by deleting duplicate data and non-conforming data (e.g., address internal segmentation repeats, including non-Chinese characters). After pre-processing, 1,552,532 addresses were used. The longest address has 54 tokens, and the average number of tokens in all the addresses is 22.88 tokens. Some examples of the processed address data are shown in Table 1.

The test data contain many addresses at the same location but with different names. The reasons are as follows: 1. It is same location but has a different address name. 2. Limited by the accuracy of GPS data, some addresses are incorrectly placed at the same location.

4.2. Address language model pre-training

This section examines the semantic feature-extraction effect through the accuracy of the MLM task mentioned earlier. To determine the influence of the number of internal encoders in the ALM on the result of the MLM task, we set 6, 8, 10, and 12 internal encoders to compare the accuracy and computation time and finalize the number of internal encoders.

Only training and validation sets are required to pre-train the model. Due to the large address corpus, we set the proportions of training set and verification set as approximately 99% (1537532) and 1% (15000), respectively. In terms of hyper-parameter setting, the dictionary size is 9431, including Chinese characters and numbers; the maximum sentence length is set to 55; the hidden size is 768; and the

Table 1
Examples from the test data.

Address (in Chinese)	Address (in English)	Longitude	Latitude	Community (in Chinese)	Community (in English)
浙江省杭州市上城区凯旋门公寓2幢3单元2201室	Room2201,Unit3,Building2, KaixuanmenApartment, ShangchengDistrict,Hangzhou,Zhejiang	120.181583	30.258083	老浙大社区	LaozhedaCommunity
浙江省杭州市上城区老复兴街2号白塔公园古一宏禅茶馆	GuyihongchanTeahouse,BaitaPark, No.2,LaoFuXingStreet, ShangchengDistrict,Hangzhou,Zhejiang	120.1413490	30.199027	白塔岭社区	BaitalingCommunity
浙江省杭州市上城区延安路1号肯德基店	KFC,No.1Yan'anRoad, ShangchengDistrict,Hangzhou,Zhejiang	120.163825	30.241045	清河坊社区	QinghefangCommunity

number of self-attention heads is 12. The remaining hyper-parameters are shown in Table 2, which describes the differences between the hyperparameters for the ALM, the GSAM and the address location prediction model. We have found that most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. In addition, all the training in the paper was done on a single GPU. Our experiments were performed on a platform with a CPU of Intel(R) Xeore(R) E5–2650 and a GPU of Titan xp whose global memory size is 12 GB.

The variation in the training loss value of different encoders is shown in Fig. 6. Because the large fluctuations in the loss value affect the visual effect, we smoothed the curve value by 0.9. The loss value decreases rapidly before 40k steps and then slowly decreases until it reaches a gentle oscillation. Moreover, the position differences between the four curves is not significant, which proves that they have a similar performance. The index values for the four models are shown in Table 3. It can be seen that the difference in accuracies between the four models is small, which indicates that increasing the number of encoders can improve the performance, but not significantly. Besides, the 12 encoders model spends additional about 5 h of training time than the six encoders model. Given these two considerations, we suggest that researchers refer to the situation when the model uses six or fewer internal encoders in the future. Of course, if it is possible to use multiple GPUS or Cloud TPU for parallel training, the model containing more internal encoders can be considered to improve the accuracy. In our research, we will continue to use models with 12 internal encoders. Because the objective task accuracy of this model is the highest in all cases, it can provide the most positive influence for our follow-up experiments.

The model with 12 internal encoders has the highest accuracy, and the time burden from the actual training is within an acceptable range. Therefore, we choose the model with 12 internal encoders to better obtain the semantic features of addresses.

On the other hand, the accuracy of the MLM tasks for all four models is in the range of 90% to 91%, which seems low, so we did some follow-up experiments to determine the reason for this result. By replacing all the numbers in the address corpus with a common identifier “NUM”, the accuracy of the 12 internal encoders model reached 97.75%. The results show that the number in the character prediction task will interferes with the fitting of model parameters, which affect the precise semantic expression. Because some numbers are not unique, they can be in multiple attributes (e.g., road numbers, building numbers, room numbers). Therefore, an accuracy of 90.79% reaches the expected goal of our study. Numbers in addresses will have a greater influence after the fusion with the coordinates, and much spatial information will be lost after the replacement of all numbers with “NUM”, so we keep the original numbers in the addresses without processing them.

4.3. Geospatial-semantic fusion clustering

In this study, functional areas (e.g., residential quarters, commercial areas, and key roads) are used as references to determine the number of clusters. The pre-experiment is carried out in “Shimutian Community” as an example, and the functional areas in this community can be divided into 13 parts, which are shown in panel on the left in Fig. 7. The clustering results, which are shown in the panel on the right, fit well within the division of functional areas in the panel on the left. Accordingly, we set the number of clusters to 500 since the experimental area contains 54 communities and the total number of functional areas is approximately 500.

We use four cases with geospatial-semantic weight ratios of 0.4/0.6, 0.5/0.5, 0.6/0.4 and 0.7/0.3 to study and compare the clustering results. The clustering effect is judged by whether the 770-dimensional feature clustering results conform to the artificial division regions in the two-dimensional map. We observe a typical area in “Shimutian community” in the experimental area, as shown in Fig. 8.

From the perspective of the entire region, strong geospatial clustering exists in all cases, which proves that geospatial weight plays a significant role in clustering. By comparing the typical areas in “Shimutian Community”, the geospatial-semantic ratio of 0.6/0.4 results in more consistent geospatial divisions in “Min'anyuan(RQ)” (dark orange points) and “Shuichenghuayuan(RQ)” (dark purple points), which are better than the other three results. In addition, the 0.6/0.4 result can identify road network elements. Taking “Fuxing Road” (gold in Fig. 7 and dark grey in the 0.6/0.4 results) in the area as an example; its corresponding addresses all contain “ Fuxing Road No.XXX”. In consideration of spatial constraints, the 0.6/0.4 result also does not classify the entire road into one class (e.g., dark blue points and green points near dark grey points). Finally, by comparing the functional areas in Fig. 7, the 0.6/0.4 result is the most consistent. For example, the red clustering points correspond to “Haowangjiaogongyu(RQ)”, and the light grey points correspond to “Fuxingjiayuan(RQ)”.

In general, the excessive setting of semantic weights will result in a discrete geospatial effect after clustering (e.g., the green points in the typical area in the 0.4/0.6 result), which will further reduce the geospatial precision of the GSAM instance. However, if the geospatial weight is too large, the semantic information of address will be ignored (e.g., in the 0.7/0.3 result). The clustering algorithm will lose its ability to divide the urban functional region, thus reducing the semantic understanding ability of the GSAM instance. The appropriate semantic and geospatial weight ratio can better cluster the high-dimensional features of the semantic geospatial fusion of addresses than can inappropriate weight ratios, and the clustering result is compatible with the geospatial information as well as human understanding and experience.

4.4. Fine-tuning

This section uses the 0.6/0.4 clustering result to fine-tune the ALM. To check whether there is overfitting and obtain the optimal evaluation value, the evaluation set is tested every 500 steps during training. The evaluation set contains 15,000 addresses, and the overall gradient curve of the training process is shown in Fig. 9.

In Fig. 9, the training loss oscillates after 100k steps and is within the range of 0.05 to 0.08 after smoothing. The evaluation loss value is slightly different from the training loss, indicating that there is slight overfitting. Based on the above situation, we decide to reduce the overfitting gap by as much as possible within the scope of guaranteeing the model training effect. Moreover, since the evaluation curve is basically stable within the range after 100k steps, it will not affect the effects of the model. Finally, the model instance checkpoints in step 115.5k are selected as the GSAM's generation instance. Performances of the GSAM's instances are shown in Table 4.

From Table 4, the values of three evaluation metrics are all above 0.96, indicating that the GSAM can accurately identify clustering labels corresponding to addresses. However, although the performance of the ALM cannot be directly compared with that of the GSAM, the target task accuracy of the ALM is 0.9079 and the GSAM's classification accuracy is above 0.97, which indirectly indicates that the problem with numbers in the addresses can be identified by GSAM. Only addresses with different road numbers, building numbers, and room numbers will be classified into different clusters due to geospatial weight constraints. This finding further illustrates that the GSAM can assign geospatial features of clustering granularity to numbers in addresses and successfully integrates semantic and geospatial features.

Table 2
Hyper-parameter comparison for all models.

	ALM	GSAM	Address location prediction model
Batch size	32	64	64
Optimizer	Adam		
lr	2.00E-05	5.00E-05	1.00E-04
Epochs	3	4	5
Train_interval_eval	–	500	500

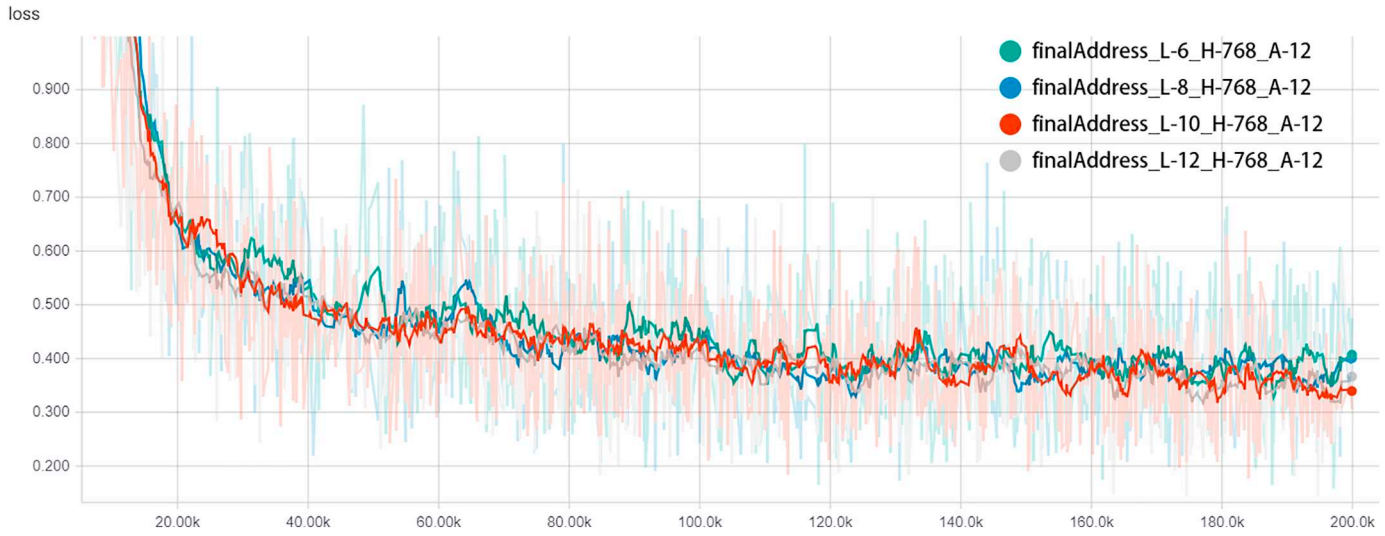

Fig. 6. Comparison of the training gradient curves with 6, 8, 10, and 12 internal encoders. The abscissa represents the training step size; the ordinate represents the loss value.

Table 3
Comparison of the training performances under different encoders of the ALM.

Number of encoders	Training time	Training loss	Mean evaluation loss	Accuracy
6	5 h, 59 m, 28 s	0.4029	0.3703	90.41%
8	7 h, 32 m, 30 s	0.3889	0.3632	90.52%
10	9 h, 25 m, 30 s	0.3578	0.3565	90.63%
12	11 h, 10 m, 41 s	0.3423	0.3509	90.79%

4.5. Address location prediction

This section constructs three control models. The control variable of the three models is the upstream part of the model, while the downstream part is the network structure constructed for the address location prediction. The upstream part of the two control models is the pre-trained ALM, but one of the upstream part is frozen and the other is not. They are used to compare the direct effects of the ALM and whether it is frozen on the coordinate-prediction task. The other upstream part is based on the word2vec model. Since word2vec is a static feature representation model, we first generate the feature representation by word2vec for all characters in the address corpus. The downstream network's structure of all models is consistent. Then, we use average pooling to generate the sentence representation of each address as the input of the downstream network's structure for training.

We define the dimension size of the hidden layer as variable and set it to 768,1024, 2048 and 4096 for training and comparison. We use the same metrics as a previous work in text-based geographical location inference: the mean and median error distances between the predicted location and the actual location (Cheng, Caverlee, & Lee, 2010). We also use the minimum absolute error. The performances of each model are shown in Table 5.

First, we compared the performances of the four models. Obviously, the GSAM-based model consistently performs better than the other three models under any hidden layer dimension. On the other hand, the results show that the regression task that directly predicts coordinates through the purely ALM-based models performs poorly. The model is required to fully fit the optimal curve related to the regression task, which is difficult. However, compared to the other models, the GSAM can better fit the clustering divisions after prior fine-tuning, which greatly helps to fit the regression curve.

Next, we compare the influence of freezing the upstream part of the model, which are ALM(unfreeze) and ALM(freeze) in Table 5. The results show that the performance based on the frozen ALM is much better than that of the unfrozen ALM, which verifies our hypothesis in Section 3.4. Therefore, we freeze the other two models' upstream structure (i.e., GSAM-based and word2vec-based). In addition, because all the model parameters are adjusted when unfreezing the upstream structure and the structure of the downstream model is weaker than the

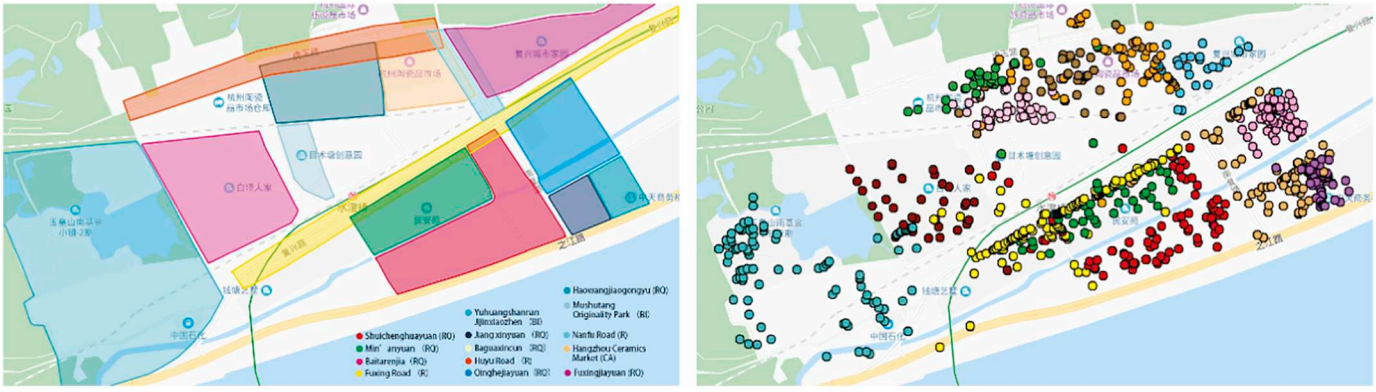

Fig. 7. Clustering results comparison in the pre-experiment. In the legend, the abbreviations in parentheses represent different functional area types, where RQ stands for residential quarter, R stands for road (i.e., many places are indicated by the XXX Road No. XXX), BI stands for business incubator, and CA stands for commercial area.

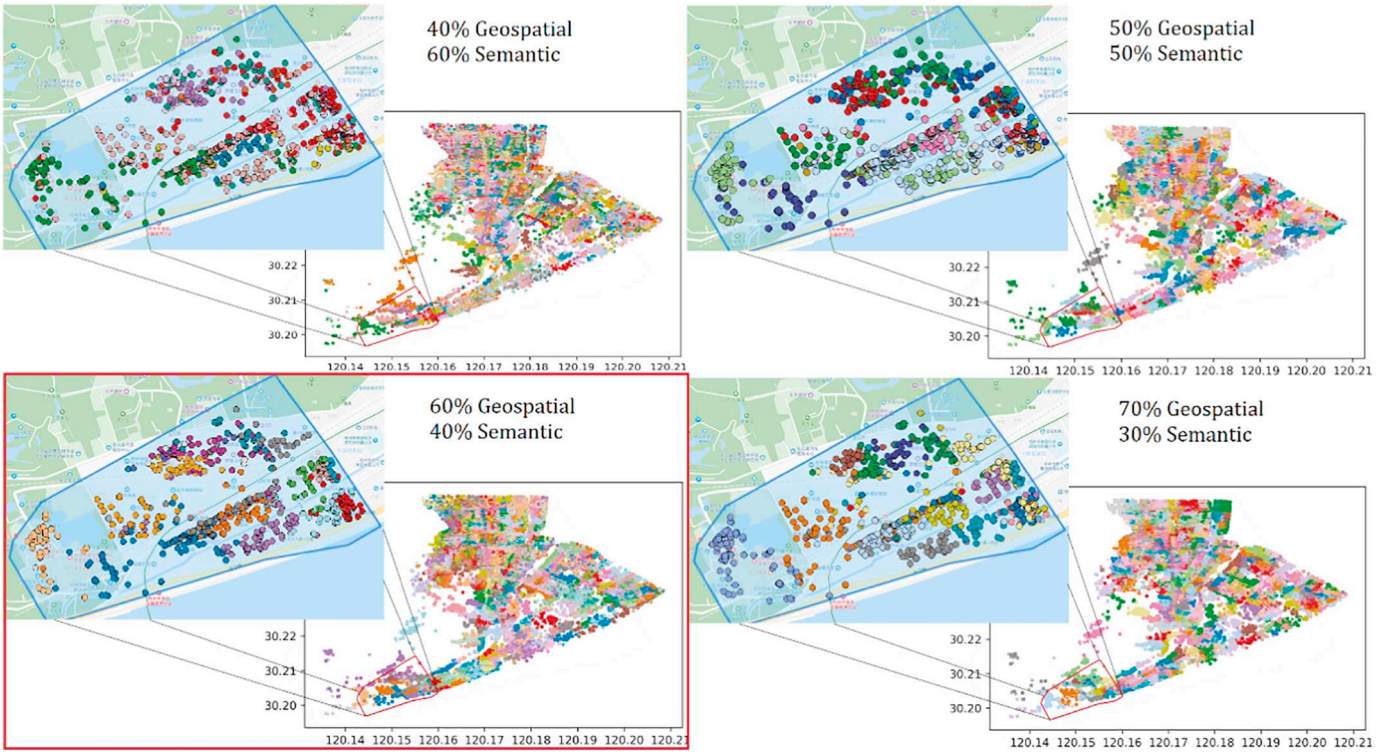

Fig. 8. Clustering results under different geospatial-semantic weight ratios. The figure with red border is the clustering result we use in the final model. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

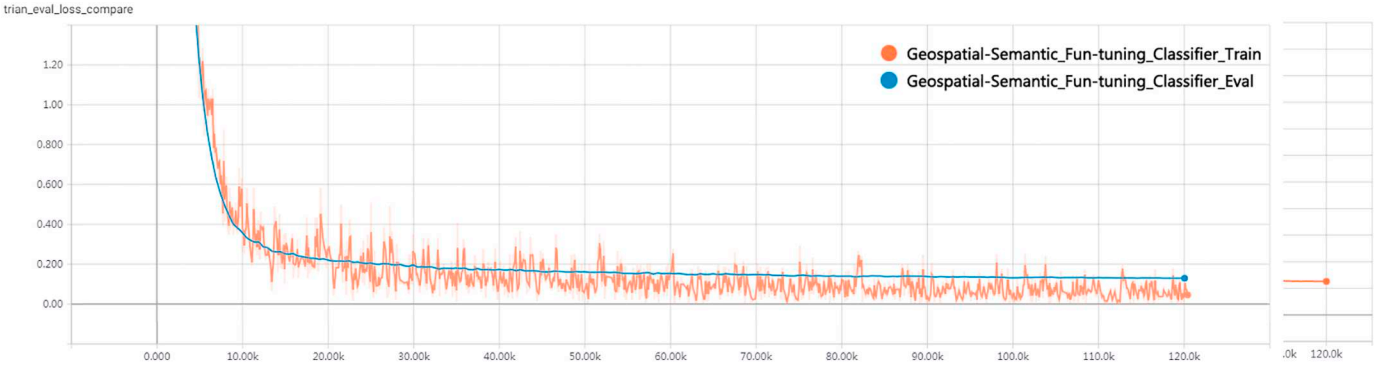

Fig. 9. Overall gradient curve of the training process. The blue line represents the evaluation curve, and the orange line represents the training curve. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 4
Performances of the GSAM.

	Training loss	Evaluation loss	Evaluation accuracy	Evaluation micro F1	Evaluation macro F1
GSAM	0.1280	0.1176	0.9706	0.9710	0.9607

structure of the upstream model, the hidden layer size of the downstream model has little influence on the result.

Then, we discuss the performances of the ALM and word2vec. It is obvious that word2vec, as an upstream structure, works better. In our opinion, compared to the fixed character representation from word2vec, the contextual character representation from the ALM reflects the dynamics of its semantic output to achieve the expression effect of high semantic features. However, this will also result in the discrete distribution of the representation vectors in high-dimensional space. Therefore, it is difficult for the target neural networks in our experiment to achieve a better fit on the representations of addresses, resulting in high error values. The word2vec-based model is better overall and as the downstream neural network widens, but its fatal flaw is that it cannot be fine-tuned in advance based on the clustering results.

Finally, we use the GSAM-based model to compare the training gradient curve and the evaluation metric under four different hidden layer sizes. The comparison between the training and evaluation gradient curves is shown in Fig. 10. The two curves maintain a high degree of overlap in different hidden layer dimensions, indicating that there is no overfitting or underfitting. On the other hand, with the increase in the hidden layer dimension in the target task module, the fitting ability of the model is gradually enhanced. We believe that for the coordinate-prediction task, the wider neural network can map features to a higher dimensional space so that the subsequent neural network can carry out the nonlinear transformation of parameter variables, enrich the output results, and enable the model to have a more accurate fitting ability for the target values.

Table 5
Performances of each model in the coordinates-prediction task.

Models	Hidden size	Geolocation-error(meters)		
		Mean	Mid	Min
GSAM-based	768	294.3	243.5	3.603
	1024	270.3	220.5	1.714
	2048	227.7	181.2	1.159
	4096	194.5	149.0	1.118
ALM(freeze)-based	768	1285.0	1131.2	9.204
	1024	1248	1106	9.957
	2048	1190.0	1046.0	13.750
	4096	1170.0	1021	6.166
ALM(unfreeze)-based	768	1725.0	1631.1	8.220
	1024	1731.8	1685.9	6.228
	2048	1732.0	1626.1	7.46
	4096	1730.0	1623.3	25.487
Word2Vec-based	768	762.1	602.1	2.453
	1024	690.6	527.6	3.005
	2048	524.7	368.9	3.300
	4096	445.1	298.9	2.058

5. Conclusions and future work

In this research, we construct the geospatial semantic address model (GSAM), which can extract the computational representations of addresses and use it to predict the location of address. Aiming at the characteristic expression of geographical address locations, we also propose a feature fusion solution for addressing semantic and geospatial features. Besides, a weighted clustering method is designed to obtain the results of strong correlation with fine-grained urban neighbourhood area division. Moreover, an appropriate neural network architecture is obtained by constructing the downstream task of address location prediction for reference of more applications in the future.

We come to the following conclusions for our research. Firstly, ALM based on Transformer encoders with multi-head self-attention can well capture the semantic information of addresses, which can be used to extract the semantic computational representations. Secondly, the clustering results after feature fusion are the most consistent with the results of fine-grained urban neighbourhood division when the weight ratio of geospatial and semantic features is 0.6/0.4. Furthermore, the GSAM can accurately identify clustering labels and the values of evaluation metrics are all above 0.96, indicating that the computational representation extracted by GSAM is well integrated with geospatial and semantic information. Moreover, through the downstream task comparison experiments of address location prediction, we find that the error of GSAM-based address location prediction is the smallest under the condition of three-layer fully connected neural network and hidden layer size of 4096. it is found that better results can be obtained by constructing the three-layer fully connected neural network and the hidden layer size reaches a thousand level. The error accuracy of the address location prediction task is much better than that of purely ALM-based and word2vec-based, which further proves the validity of our model.

This study also has some limitations. First, although the address corpus in our study has a million level, the research region is not large enough. Second, when geospatial and semantic representation are fused, the weight ratio of the two is taken as a span by 10%, and more comparison experiments can be conducted with a finer-grained weight ratio of the span. Finally, in order to compromise the geographical distance measurement, we use Euclidean distance as the metric in the fusion clustering, but Euclidean distance is not the best measurement metric for semantic similarity measurement.

In all, representation learning for addresses is an emerging field that

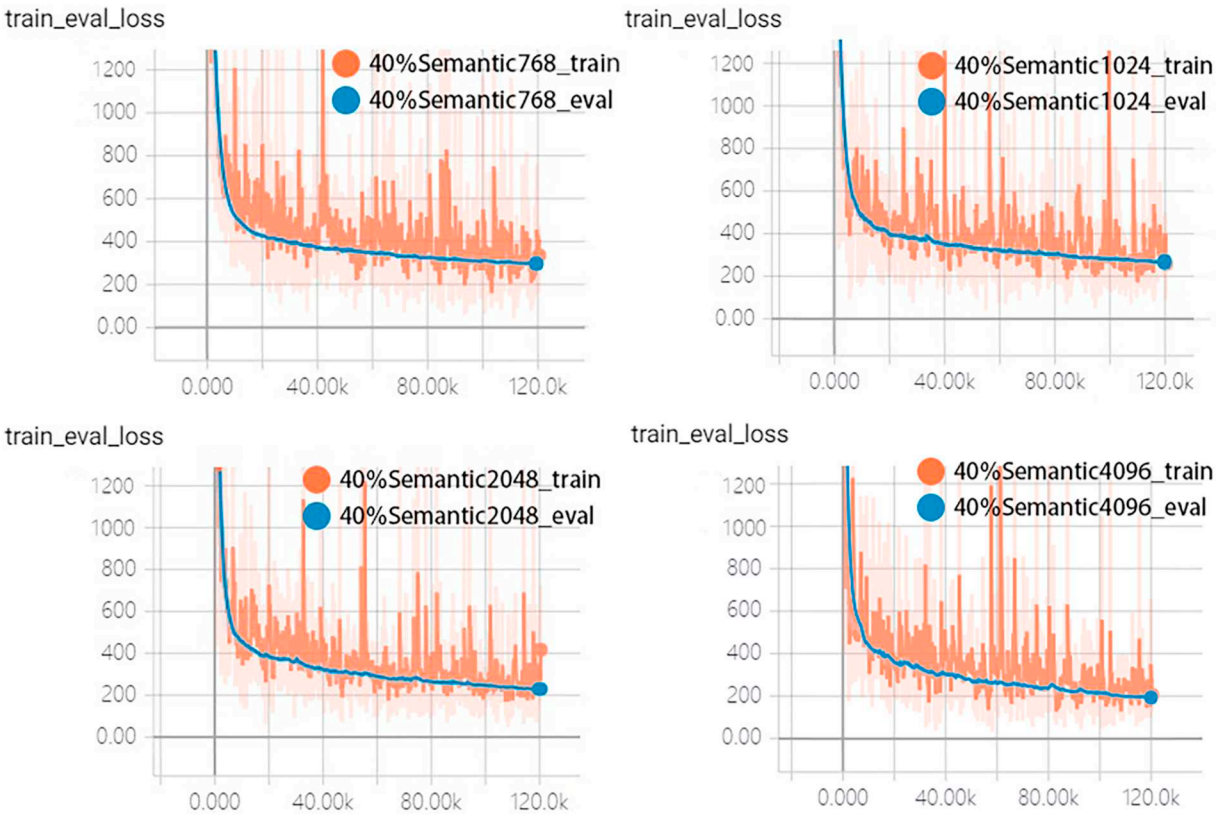

Fig. 10. Gradient curve comparison under different hidden sizes. The blue line represents the evaluation curve, and the orange line represents the training curve. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

can not only improve our understanding of addresses, but also provide new ways to use addresses as input into deep learning models. By establishing the mapping relationship between addresses and geospatial entity knowledge, an ALM with cognitive abilities to address entity elements can be constructed to expand the content of address features and realize the further understanding of information in future studies.

Acknowledgments

This research was supported by the Ministry of Science and Technology of the People's Republic of China under Grant 2018YFB0505000 and the National Natural Science Foundation of China (41871287, 41671391).

Appendix A. High-res images of geospatial-semantic fusion clustering results

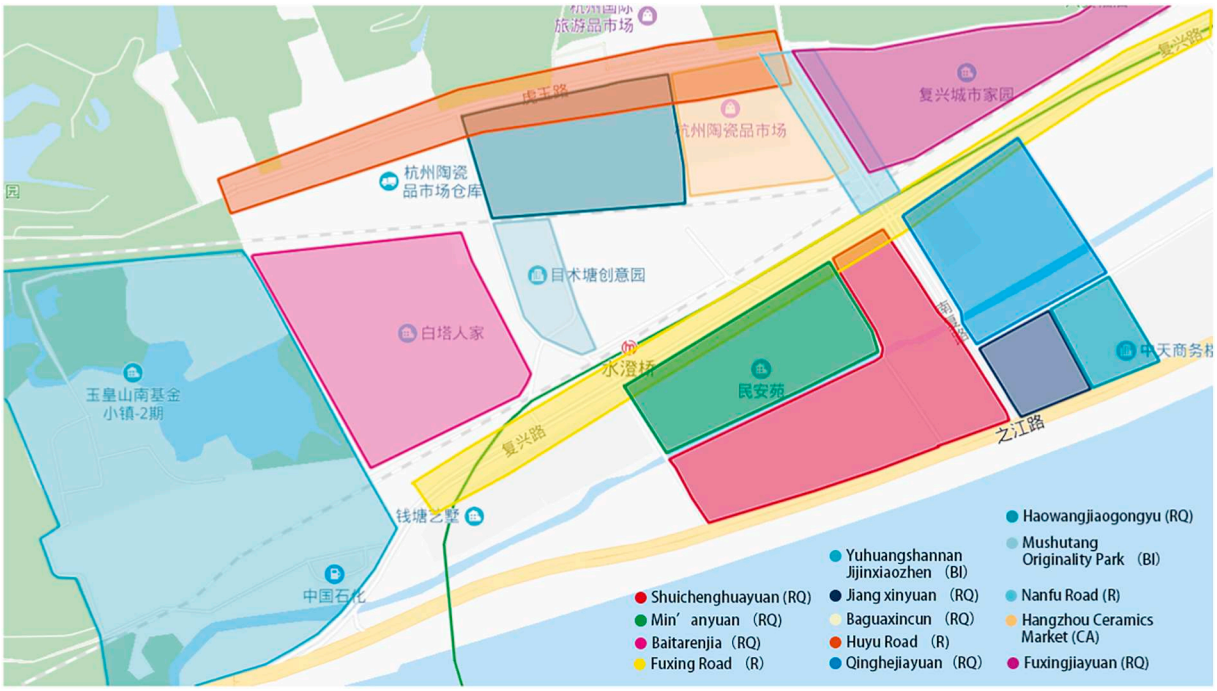

Fig. A.1. The high-res image of the left panel in Fig. 7.

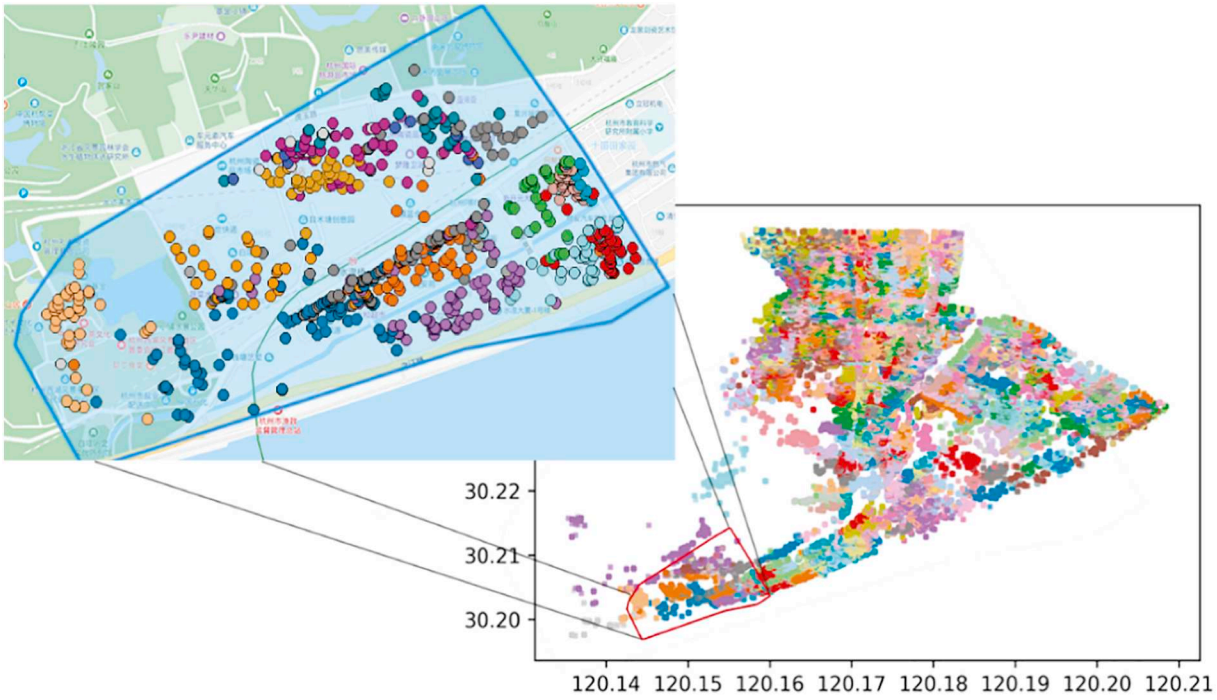

Fig. A.2. The high-res image of the figure with a red border in Fig. 8. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

References

Arthur, D., & Vassilvitskii, S. (2007). k-means+ +: The advantages of careful seeding. Society for Industrial and. *Applied Mathematics,* 1027–1035.

Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research, 3*(Feb), 1137–1155.

Cayo, M. R., & Talbot, T. O. (2003). Positional error in automated geocoding of residential addresses. *International Journal of Health Geographics, 2*(1), 10.

Cheng, Z., Caverlee, J., & Lee, K. (2010). *You are where you tweet: A content-based approach to geo-locating twitter users.* ACM759–768.

Cocos, A., & Callison-Burch, C. (2017). *The language of place: Semantic value from geospatial context.* 99–104.

Conedera, M., Vassere, S., Neff, C., Meurer, M., & Krebs, P. (2007). Using toponymy to reconstruct past land use: A case study of "brüsáda"(burn) in southern Switzerland. *Journal of Historical Geography, 33*(4), 729–748.

Dai, A. M., & Le, Q. V. (2015). Semi-supervised sequence. *learning,* 3079–3087.

DeLozier, G., Baldridge, J., & London, L. (2015). *Gazetteer-independent toponym resolution using geographic word profiles*.

Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2018). *Bert: Pre-training of deep bidirectional transformers for language understanding, arXiv preprint arXiv:1810.04805*.

Elkan, C. (2003). *Using the triangle inequality to accelerate k-means.* 147–153.

Er, M. J., Zhang, Y., Wang, N., & Pratama, M. (2016). Attention pooling-based convolutional neural network for sentence modelling. *Information Sciences, 373*, 388–403.

Feng, S., Cong, G., An, B., & Chee, Y. M. (2017). *Poi2vec: Geographical latent representation for predicting future visitors*.

Goldberg, D. W., Wilson, J. P., & Knoblock, C. A. (2007). From text to geographic coordinates: The current state of geocoding. *URISA Journal, 19*(1), 33–47.

Guo, H., Zhu, H., Guo, Z., Zhang, X., & Su, Z. (2009). *Address standardization with latent semantic association.* ACM1155–1164.

Hao, P., Cheang, W., & Chiang, J. (2019). Real-time event embedding for POI recommendation. *Neurocomputing, 349*, 1–11.

Howard, J., & Ruder, S. (2018a). *Universal language model fine-tuning for text classification, arXiv preprint arXiv:1801.06146*.

Howard, J., & Ruder, S. (2018b). *Fine-tuned language models for text classification, arXiv preprint arXiv:1801.06146*.

Jones, C. B., & Purves, R. S. (2009). Geographical information retrieval. *Encyclopedia of Database Systems,* 1227–1231.

Ju, Y., et al. (2016). *Things and strings: Improving place name disambiguation from short texts by combining entity co-occurrence with topic modeling.* Springer353–367.

Kalchbrenner, N., Grefenstette, E., & Blunsom, P. (2014). *A convolutional neural network for modelling sentences, arXiv preprint arXiv:1404.2188*.

Kim, J., Vasardani, M., & Winter, S. (2017). Similarity matching for integrating spatial information extracted from place descriptions. *International Journal of Geographical Information Science, 31*(1), 56–80.

Laere, O. V., Schockaert, S., Tanasescu, V., Dhoedt, B., & Jones, C. B. (2014). Georeferencing Wikipedia documents using data from social media sources. *ACM Transactions on Information Systems (TOIS), 32*(3), 12.

Liu, J., & Inkpen, D. (2015). *Estimating user location in social media with stacked denoising auto-encoders.* 201–210.

Liu, X., Andris, C., & Rahimi, S. (2019). Place niche and its regional variability: Measuring spatial context patterns for points of interest with representation learning. *Computers, Environment and Urban Systems, 75*, 146–160.

MacQueen, J. (1967). *Some methods for classification and analysis of multivariate observations. Oakland, CA, USA*. 281–297.

Matci, D. K., & Avdan, U. (2018). Address standardization using the natural language process for improving geocoding results. *Computers, Environment and Urban Systems, 70*, 1–8.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). *Distributed representations of words and phrases and their compositionality.* 3111–3119.

Özay, M., & Vural, F. T. Y. (2010). Analysis of feature concatenation operation on vector spaces. *IEEE,* 1–4.

Peters, M. E., et al. (2018). *Deep contextualized word representations, arXiv preprint arXiv:1802.05365*.

Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). Improving language understanding by generative pre-training. *https://s3-us-west-2.amazonaws.com/openai-assets/research-covers/languageunsupervised/languageunderstandingpaper.pdf*.

Roller, S., Speriosu, M., Rallapalli, S., Wing, B., & Baldridge, J. (2012). Supervised text-based geolocation using language models on an adaptive grid. *Association for Computational Linguistics,* 1500–1510.

Sculley, D. (2010). *Web-scale k-means clustering.* ACM1177–1178.

Tan, M., Dos Santos, C., Xiang, B., & Zhou, B. (2016). *Improved representation learning for question answer matching.* 464–473.

Tan, M., Santos, C. D., Xiang, B., & Zhou, B. (2015). *LSTM-based deep learning models for non-factoid answer selection, arXiv preprint arXiv:1511.04108*.

Tian, Q., et al. (2016). Using an optimized Chinese address matching method to develop a geocoding service: A case study of Shenzhen, China. *ISPRS International Journal of Geo-Information, 5*(5), 65.

Vaswani, A., et al. (2017). *Attention is all you need.* 5998–6008.

Wasserman, L. (2004). *A concise course in statistical inference. 43*, Heidelberg: Springer97–99.

Wing, B. P., & Baldridge, J. (2011). Simple supervised document geolocation with geodesic grids. *Association for Computational Linguistics,* 955–964.

Yan, B., Janowicz, K., Mai, G., & Gao, S. (2017). *From ITDL to Place2Vec: Reasoning about place type similarity and relatedness by learning embeddings from augmented spatial contexts.* ACM35.

Zhai, W., et al. (2019). Beyond Word2vec: An approach for urban functional region extraction and identification by combining Place2vec and POIs. *Computers, Environment and Urban Systems, 74*, 1–12.

Zheng, Q., Zhu, J., Li, Z., Pang, S., & Wang, J. (2019). *Feature concatenation multi-view subspace clustering, arXiv preprint arXiv:1901.10657*.

Zhou, X., & Bhanu, B. (2008). Feature fusion of side face and gait for video-based human identification. *Pattern Recognition, 41*(3), 778–795.

Zhu, F., Zhao, T., Liu, Y., & Zhao, Y. (2018). Research on Chinese address resolution model based on conditional random field. *Journal of Physics: Conference Series, 1087*, 52040.